AI Engineer Interview Questions & Answers

50 Most Common Questions for 1-Year-Experience AI Engineers

Coding • Logic & DSA • Machine Learning • Deep Learning • NLP/LLM • MLOps • Behavioral

Focused on Remote-Based Companies Hiring from Karachi, Pakistan

---

This guide compiles the questions most frequently asked of AI/ML engineers with around one year of experience in interviews for remote roles (freelance platforms, international startups, and product companies hiring from Pakistan). Each answer is written at the depth expected at your experience level - concise, correct, and practical. Practice explaining these out loud, and be ready to relate answers back to your own projects.

A. Python & Coding Questions

1. What is the difference between a list, tuple, and set in Python?

Answer: A list is mutable and ordered ([]); a tuple is immutable and ordered (()); a set is mutable, unordered, and stores only unique elements ({}). Lists/tuples allow duplicates and indexing; sets are used for fast membership tests and removing duplicates.

2. Write a function to check if a string is a palindrome.

```
def is_palindrome(s: str) -> bool:
    s = s.lower().replace(' ', '')
    return s == s[::-1]

print(is_palindrome('Was it a car or a cat I saw'))  # True
```

3. Explain *args and **kwargs.

Answer: *args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict. They let a function accept a variable number of inputs, e.g. def f(*args, **kwargs).

4. Write a function to find duplicate elements in a list.

```
def find_duplicates(nums):
    seen, dup = set(), set()
    for n in nums:
        if n in seen:
            dup.add(n)
        seen.add(n)
    return list(dup)

print(find_duplicates([1,2,3,2,4,1]))  # [1, 2]
```

5. What is the difference between deep copy and shallow copy?

Answer: A shallow copy (copy.copy) creates a new object but nested objects still reference the originals. A deep copy (copy.deepcopy) recursively copies nested objects too, so changes to the copy never affect the original.

6. What are Python decorators? Give a simple example.

```
def timer(func):
    import time
    def wrapper(*a, **kw):
        t0 = time.time()
        result = func(*a, **kw)
        print(f'{func.__name__} took {time.time()-t0:.4f}s')
        return result
    return wrapper

@timer
def slow():
    return sum(range(10**6))
```

7. Difference between == and is in Python?

Answer: == compares values for equality. is compares object identity (whether both references point to the same memory location). Two equal lists can be == True but is False if they are different objects.

8. How does Python manage memory, and what is the GIL?

Answer: Python uses reference counting plus a cyclic garbage collector to reclaim memory. The Global Interpreter Lock (GIL) allows only one thread to execute Python bytecode at a time, which limits true parallelism for CPU-bound multi-threaded code (use multiprocessing instead).

9. Write a function to flatten a nested list.

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

print(flatten([1, [2, [3, 4], 5], 6]))  # [1,2,3,4,5,6]
```

10. What is the difference between NumPy arrays and Python lists?

Answer: NumPy arrays are homogeneous, contiguous in memory, and support vectorized operations, making them far faster for numerical computation. Python lists are heterogeneous and general-purpose but slower for large-scale math because operations run through Python's interpreter loop instead of optimized C code.

B. Data Structures, Algorithms & Logic-Based Questions

11. Reverse a linked list (explain the approach).

Answer: Iterate through the list keeping three pointers: prev, curr, next. At each node, save next, point curr.next to prev, then move prev and curr forward one step. Repeat until curr is None; prev becomes the new head. Time O(n), space O(1).

12. What is time complexity? Give Big-O of common operations.

Answer: Time complexity describes how runtime grows with input size n. Examples: array index access O(1), linear search O(n), binary search O(log n), sorting (merge/quick avg) O(n log n), nested loops O(n^2), hash map lookup O(1) average.

13. How would you find the two numbers in an array that sum to a target?

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        need = target - n
        if need in seen:
            return [seen[need], i]
        seen[n] = i
    return None

print(two_sum([2,7,11,15], 9))  # [0, 1]
```

14. Explain recursion with an example (factorial).

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))  # 120
```

15. Classic logic puzzle: You have 8 balls, one is heavier. Using a balance scale only twice, find the heavier ball. How?

Answer: Split into 3, 3, 2. Weigh the two groups of 3 first: if balanced, the heavy ball is among the remaining 2, weigh those to find it. If unbalanced, take the heavier group of 3, weigh 2 of them: if balanced the third is heavy, else the heavier side wins. Two weighings suffice.

16. What is the difference between a stack and a queue? Give a real use case for each.

Answer: A stack is LIFO (last in, first out) - used for undo operations, DFS traversal, and function call management. A queue is FIFO (first in, first out) - used for task scheduling, BFS traversal, and request handling in servers.

17. Given a list of numbers, find the missing number from 1 to n.

```
def missing_number(nums, n):
    expected = n * (n + 1) // 2
    return expected - sum(nums)

print(missing_number([1,2,4,5], 5))  # 3
```

18. A classic logic question: If a train leaves station A at 60 km/h and another leaves station B (300 km away) at 90 km/h toward each other, when do they meet?

Answer: Combined closing speed = 60 + 90 = 150 km/h. Time to meet = distance / speed = 300 / 150 = 2 hours. This tests basic rate-time-distance reasoning, common in aptitude-style rounds.

C. Machine Learning Fundamentals

19. What is the difference between supervised, unsupervised, and reinforcement learning?

Answer: Supervised learning trains on labeled input-output pairs (classification/regression). Unsupervised learning finds structure in unlabeled data (clustering, dimensionality reduction). Reinforcement learning trains an agent to take actions in an environment to maximize cumulative reward through trial and error.

20. What is overfitting and how do you prevent it?

Answer: Overfitting is when a model learns noise/specifics of training data and performs poorly on unseen data. Prevent it with more training data, regularization (L1/L2), dropout, cross-validation, early stopping, simpler models, or data augmentation.

21. Explain the bias-variance tradeoff.

Answer: Bias is error from overly simplistic assumptions (underfitting); variance is error from sensitivity to small fluctuations in training data (overfitting). Total error = bias^2 + variance + irreducible error, so model complexity must be tuned to balance the two.

22. What is the difference between L1 and L2 regularization?

Answer: L1 (Lasso) adds the sum of absolute weights to the loss and can shrink some weights exactly to zero, producing sparse models useful for feature selection. L2 (Ridge) adds the sum of squared weights, shrinking weights smoothly toward zero without eliminating them.

23. How do you handle imbalanced datasets?

Answer: Techniques include resampling (oversampling minority class with SMOTE, undersampling majority class), class-weighted loss functions, using metrics like F1/precision-recall/AUC instead of accuracy, and ensemble methods designed for imbalance.

24. Explain precision, recall, and F1-score.

Answer: Precision = TP / (TP + FP), the fraction of predicted positives that are correct. Recall = TP / (TP + FN), the fraction of actual positives correctly found. F1 = harmonic mean of precision and recall, useful when you need a balance between the two, especially on imbalanced data.

25. What is cross-validation and why is it used?

Answer: Cross-validation (e.g., k-fold) splits data into k parts, trains on k-1 folds and validates on the remaining fold, repeating k times and averaging results. It gives a more reliable estimate of model performance than a single train/test split and helps detect overfitting.

26. What is the difference between bagging and boosting?

Answer: Bagging (e.g., Random Forest) trains multiple models in parallel on bootstrapped samples and averages/votes to reduce variance. Boosting (e.g., XGBoost, AdaBoost) trains models sequentially, each correcting the errors of the previous one, reducing bias.

27. How does gradient descent work?

Answer: Gradient descent iteratively updates model parameters in the direction opposite to the gradient of the loss function with respect to those parameters, scaled by a learning rate, until the loss converges to a minimum. Variants include batch, stochastic (SGD), and mini-batch gradient descent.

28. What is feature engineering, and can you give examples relevant to your work?

Answer: Feature engineering is transforming raw data into inputs that better represent the underlying problem to the model. Examples: encoding categorical variables (one-hot/label encoding), scaling/normalizing numeric features, creating interaction terms, extracting date parts, or text vectorization (TF-IDF, embeddings).

D. Deep Learning

29. What is a neural network, and what role does an activation function play?

Answer: A neural network is layers of interconnected nodes (neurons) that learn weighted combinations of inputs. Activation functions (ReLU, sigmoid, tanh, softmax) introduce non-linearity so the network can learn complex patterns beyond what a linear model could represent.

30. What is backpropagation?

Answer: Backpropagation computes the gradient of the loss function with respect to each weight by applying the chain rule backward through the network, layer by layer. These gradients are then used by an optimizer (e.g., SGD, Adam) to update the weights.

31. Explain vanishing and exploding gradients.

Answer: In deep networks, gradients can shrink toward zero (vanishing) or grow uncontrollably (exploding) as they backpropagate through many layers, especially with sigmoid/tanh activations. Solutions include ReLU activations, batch normalization, gradient clipping, residual connections, and careful weight initialization.

32. What is the difference between CNN and RNN, and when would you use each?

Answer: CNNs use convolutional filters to capture local spatial patterns, ideal for images and grid-like data. RNNs (and LSTMs/GRUs) process sequential data by maintaining hidden state across time steps, suited for text, time series, or speech, though transformers have largely replaced RNNs for sequence tasks.

33. What is dropout and why is it used?

Answer: Dropout randomly deactivates a fraction of neurons during each training pass, forcing the network to not rely too heavily on any single neuron. This acts as a regularizer and reduces overfitting; it is turned off during inference.

34. What is batch normalization?

Answer: Batch normalization normalizes the inputs to a layer (zero mean, unit variance) using statistics from each mini-batch, then applies learnable scale and shift parameters. It stabilizes and speeds up training and can act as a mild regularizer.

35. Explain the difference between Adam and SGD optimizers.

Answer: SGD updates weights using the raw gradient (optionally with momentum), requiring careful learning-rate tuning. Adam adapts the learning rate per parameter using estimates of first and second moments of the gradients, generally converging faster and being more forgiving of hyperparameter choice, though sometimes generalizing slightly worse than well-tuned SGD.

36. What is transfer learning, and why is it useful with limited data?

Answer: Transfer learning reuses a model pretrained on a large dataset (e.g., ImageNet or a large text corpus) and fine-tunes it on a smaller, task-specific dataset. It saves training time and compute, and typically outperforms training from scratch when labeled data is limited.

E. NLP, LLMs & Generative AI

37. What is the Transformer architecture and why did it replace RNNs?

Answer: The Transformer uses self-attention to weigh relationships between all tokens in a sequence in parallel, instead of processing step-by-step like RNNs. This enables much faster training (parallelizable), better handling of long-range dependencies, and is the foundation of models like BERT and GPT.

38. Explain self-attention in simple terms.

Answer: For each token, self-attention computes Query, Key, and Value vectors, then scores how relevant every other token is to it (via dot products of Query and Key), and produces a weighted sum of Value vectors. This lets the model dynamically focus on the most relevant context for each token.

39. What is the difference between embeddings and one-hot encoding for text?

Answer: One-hot encoding represents each word as a sparse, high-dimensional vector with no notion of similarity between words. Embeddings represent words/tokens as dense, low-dimensional vectors learned so that semantically similar words end up close together in vector space.

40. What is Retrieval-Augmented Generation (RAG), and why is it used?

Answer: RAG retrieves relevant documents/chunks from an external knowledge base (typically via vector similarity search) and feeds them into an LLM's prompt as context before generation. It reduces hallucination and lets the model answer from up-to-date or private data without retraining.

41. What is the difference between fine-tuning and prompt engineering when adapting an LLM?

Answer: Fine-tuning updates the model's weights on a task-specific dataset, changing its behavior permanently and requiring compute and labeled data. Prompt engineering crafts inputs (instructions, examples, formatting) to guide a frozen model's output without changing weights, making it faster and cheaper to iterate.

42. What are vector databases and why are they used with LLMs?

Answer: Vector databases (e.g., Pinecone, Weaviate, FAISS, Chroma) store embeddings and support fast approximate nearest-neighbor search. They power semantic search and RAG pipelines by quickly finding text chunks whose embeddings are most similar to a query embedding.

43. What is tokenization, and why can it affect LLM performance/cost?

Answer: Tokenization splits text into subword units (tokens) that the model actually processes, using algorithms like BPE or WordPiece. Since LLM pricing and context limits are measured in tokens, and rare words split into more tokens, tokenization affects both cost and how efficiently the model uses its context window.

44. What is hallucination in LLMs, and how can you mitigate it?

Answer: Hallucination is when an LLM generates plausible-sounding but factually incorrect or unsupported content. Mitigation strategies include RAG grounding, lowering temperature, prompting the model to cite sources or say 'I don't know,' output validation/guardrails, and human review for critical use cases.

F. MLOps, Deployment & System Design

45. How would you deploy a trained ML model into production?

Answer: Common approach: wrap the model in a REST/gRPC API (e.g., FastAPI/Flask), containerize with Docker, deploy to a cloud service or Kubernetes, and add monitoring/logging. For LLM apps, this may instead mean deploying an inference endpoint plus a RAG/orchestration layer behind an API.

46. What is model drift, and how do you detect it?

Answer: Model drift happens when the statistical properties of incoming data (data drift) or the relationship between inputs and outputs (concept drift) change over time, degrading model performance. It's detected by monitoring input distributions, prediction distributions, and live performance metrics, triggering retraining when thresholds are breached.

47. What is the difference between batch inference and real-time (online) inference?

Answer: Batch inference processes large volumes of data on a schedule (e.g., nightly scoring jobs), optimized for throughput. Real-time inference responds to individual requests with low latency (e.g., a live chatbot or recommendation API), optimized for speed per request.

48. How would you design a scalable chatbot system using an LLM (high level)?

Answer: Typical design: user request -> API gateway -> orchestration layer that manages context/session -> retrieval step (vector DB) for RAG if needed -> LLM inference (with caching for repeated queries) -> post-processing/guardrails -> response. Add rate limiting, logging, and monitoring for cost/latency, and use async queues for scale.

49. What tools have you used (or would use) for experiment tracking and versioning?

Answer: Experiment tracking: MLflow, Weights & Biases. Data/model versioning: DVC, Git LFS. These let you reproduce results, compare runs, and roll back to earlier model versions - important once a project has more than one contributor or model iteration.

50. How do you monitor cost and latency when using LLM APIs (e.g., OpenAI/Anthropic) in a production app?

Answer: Track token usage and cost per request/user, cache repeated queries, set max token limits, choose the smallest model that meets quality needs, use streaming for perceived latency, and log response times/error rates on a dashboard to catch regressions early.

Bonus: Behavioral / HR Questions (Common in Remote PK Interviews)

B1. Tell me about an AI/ML project you worked on in your 1 year of experience.

Answer: Structure your answer with the STAR method: Situation (the problem/business need), Task (your role), Action (tools/models/approach you used, e.g., Python, scikit-learn, a specific model), Result (measurable outcome - accuracy improvement, time saved, or what you learned).

B2. How do you stay updated with fast-moving AI/LLM developments?

Answer: Mention concrete habits: following papers (arXiv, Papers with Code), newsletters, Hugging Face releases, hands-on side projects, and communities/Discords - showing genuine ongoing learning rather than a generic answer.

B3. Why do you want to work remotely with an international/US-based company?

Answer: Be honest and specific: exposure to global engineering practices, better compensation, learning from experienced teams, and flexibility - while emphasizing your comfort with async communication and self-management.

B4. Describe a time a model you built did not perform well. What did you do?

Answer: Walk through your debugging process: checked data quality/leakage, tried different features or architectures, evaluated with proper metrics, and iterated - showing a methodical, non-defensive approach to failure.

B5. How do you handle working across time zones with a remote team?

Answer: Talk about overlap-hour scheduling, clear async updates (written status, Loom videos), using tools like Slack/Notion/Linear, and being proactive about blockers so the team isn't waiting on you.

Quick Prep Tips

- Be ready to share your GitHub / portfolio - remote companies almost always ask for it.

- Practice explaining projects using STAR (Situation, Task, Action, Result).

- Brush up on basic Python coding on a whiteboard/shared editor - many remote interviews use CoderPad/HackerRank.

- Know the difference between classic ML and LLM-based approaches - many 2026 roles blend both.

- For MLOps/system-design questions, a simple, clearly explained architecture beats an overcomplicated one.